PresupACL

Plan de implementación — Incremento 1

Del diseño al plan: RF-02 Registro de clientes y direcciones

Cliente: ACL Alarmas y Cámaras de León
Empresa desarrolladora: SofChi
Desarrollo de Aplicaciones y Servicios Virtuales — Universidad Iberoamericana León
Profesor: Ing. Alejandro Montes Moreno

Proyecto	PresupACL — sistema local de cotizaciones de seguridad electrónica
Integrantes	Sofía Quesada — módulos de dominio y validación (clientes_registro) Vanesa Estrada — persistencia y estructura (almacenamiento_datos) Reparto acordado por el equipo el 29-oct-2026 (D11, parte de implementación).
Fecha / versión del plan	29 de octubre de 2026 · v1.2 (incorpora revisión de consistencia adicional H7-H9, aprobada por el equipo el 29-oct-2026)
Documentos base	Especificación de requerimientos, diseño modular y persistencia v2.4 (22-oct-2026)
Repositorio	Pendiente de crear (tarea T-01). Se anotará el enlace al existir.

Propósito y resumen del plan

Este plan convierte la especificación v2.4 de PresupACL en trabajo de desarrollo ordenado y verificable. No implementa el sistema completo: define qué se construirá primero, en qué módulos y archivos, en qué orden, cómo se comprobará cada avance y qué queda fuera del primer incremento.

RF elegido	RF-02 — Registrar un cliente nuevo con nombre, teléfono y una o varias direcciones (HU-02, prioridad Alta).
Módulos	almacenamiento_datos (infraestructura) y clientes_registro (dominio). Tablas: clientes y direcciones_cliente.
Tareas	Seis tareas (T-01 a T-06), con dos ramas que pueden avanzar en paralelo y se integran en T-06.
Demostración	Ejecutar las pruebas PT-02-N, PT-02-L, PT-02-E, PT-02-E2 y P01: un cliente con dos direcciones se guarda, se cierra la conexión, se vuelve a abrir y los datos siguen ahí; un registro incompleto se rechaza sin guardar nada.
Qué NO incluye	Pantallas web, login, edición (RF-03), búsqueda (RF-04), SYSCOM, cotizaciones y las otras siete tablas.

Paso 1. Insumos verificados

Antes de planear revisamos que los documentos usen los mismos identificadores. Lo que falta se marca como pendiente; no se inventa.

Documento / artefacto	Disponible	Versión / observación
Alcance y exclusiones	Sí	v2.4, Presentación del proyecto y sección 1. Fuera de alcance: nube, pagos, mantenimiento correctivo, recordatorios, roles múltiples, catálogo de técnicos, envío automático de WhatsApp.
RF y RNF priorizados	Sí	v2.4, sección 1.1: RF-01 a RF-21 y RNF-01 a RNF-09.
HU y criterios de aceptación	Sí	v2.4, sección 1.3: HU-01 a HU-19 (incluye HU-13.1); CA-01 a CA-67 con prueba PT-XX-N/L/E.
Matriz de trazabilidad	Sí	v2.4: RF → HU → CA → PT → módulo (secciones 1 y 2.6); persistencia por RF en 3.9.
Diseño modular y estructura de carpetas	Sí	v2.4, sección 2: 20 módulos en 7 bloques; estructura en 2.8 (Python propuesto, sin confirmar).
Diseño de persistencia	Sí	v2.4, sección 3: SQLite, 9 tablas, módulo almacenamiento_datos, pruebas P01–P10. Revisado por Sofía Quesada el 15-oct.
Repositorio y bitácora de IA	Parcial	Bitácora de IA: sí (3.11). Repositorio: no existe todavía → T-01.

Revisión de consistencia (Prompt 1)

#	Hallazgo	Evidencia	Impacto	Pregunta / decisión para el equipo
H1	RF-02 exige “una o varias direcciones”, pero ningún CA prueba el caso de cero direcciones.	RF-02 (1.1); CA-04, CA-38 y CA-05 (1.3).	Podría guardarse un cliente sin dirección y luego bloquear RF-16 (aceptar sin dirección).	Se agrega CA-68 con prueba PT-02-E2. Aprobado por el equipo (29-oct).
H2	CA-06 (aviso por teléfono duplicado) no tiene regla de unicidad.	Sección 1.6; D07.	No se puede programar ni aviso ni bloqueo.	Queda fuera del incremento 1. Dos clientes pueden compartir teléfono hasta que ACL decida (D07).

#	Hallazgo	Evidencia	Impacto	Pregunta / decisión para el equipo
H3	La tecnología no está confirmada.	D09; estructura 2.8 “sujeta a confirmar”.	Bloquea pantallas, sesiones y PDF.	SUPUESTO: Python 3 + sqlite3 de la biblioteca estándar + pytest. El incremento 1 no necesita framework web. Confirmado por el equipo para el incremento 1; framework web, PDF, hash y sesiones siguen abiertos en D09.
H4	El documento pide cerrar D01–D11 antes de programar.	Secciones 4.2 y 5.	Si se toma literal, no se puede iniciar.	RF-02 solo toca D07 (parcial), D08 (fechas) y D09 (parcial); se trabajan con supuestos marcados. D01–D06 y D10 no afectan este incremento.
H5	P01 está asociada a RF-02 y RF-03, pero su acción solo prueba el registro.	Sección 3.10.	Ninguno grave.	P01 se ejecuta en el incremento 1; la parte de edición (RF-03) se prueba con PT-03-* en el incremento 2.
H6	creado_en / actualizado_en requieren fecha y hora, y la zona horaria está pendiente.	Tabla clientes (3.4); D08.	Formato inconsistente si cada quien elige.	Acordado: ISO 8601 con la hora local del servidor. La zona horaria definitiva se valida con ACL en D08.
H7	El cap. 1 asigna cada RF a un módulo con la nomenclatura antigua de 11 módulos gruesos; los cap. 2-4 usan la nomenclatura nueva de 20 módulos finos, sin tabla de equivalencia.	Cap. 1: “Módulo previsto: Clientes / Cotizaciones / Documentos PDF / Comunicación”, etc. Cap. 2.5/2.6 y cap. 3 usan en cambio clientes_registro, cotizaciones_edicion, generador_pdf_cotizacion, mensajeria_whatsapp, etc.	Un lector que entra por el cap. 1 no puede verificar, sin releer el cap. 2, qué módulo fino responde a cada RF.	Decidido por el equipo (29-oct): se actualiza el cap. 1 para usar los nombres de módulo del cap. 2.5 en todo el documento, para evitar confusiones. Responsable: Sofía.
H8	Las tablas resumen 2.6 y 3.9 listan un solo módulo “principal” por RF, pero las fichas de módulo en 2.5 documentan responsabilidad compartida que la tabla resumen no siempre refleja.	Ficha de generador_pdf_cotizacion: “Requerimientos: RF-11; regeneración en RF-14” — pero la fila de RF-14 en 2.6 solo dice cotizaciones_edicion. RF-15 sí aparece con doble módulo en la misma tabla.	Tratamiento inconsistente dentro de la misma tabla; puede subestimarse qué módulos deben coordinarse al implementar RF-14.	Decidido por el equipo (29-oct): se agrega generador_pdf_cotizacion a la fila de RF-14 en las tablas 2.6 y 3.9 (como ya está en RF-15). No afecta el incremento 1 (RF-02); aplica cuando se implemente RF-14. Responsable: Sofía.
H9	Los campos “snapshot” de cotizaciones y de ordenes_servicio usan convenciones distintas sin que el documento lo señale.	cotizaciones guarda cliente_snapshot_json, direccion_snapshot_json y acl_snapshot_json como objetos JSON; ordenes_servicio guarda la misma clase de información como campos planos separados (cliente_nombre_snapshot, cliente_telefono_snap	Quien programe la capa de datos puede copiar el patrón equivocado de una tabla a otra, o dudar si ordenes_servicio debería también ser JSON.	Decidido por el equipo (29-oct): se unifican ambas tablas usando campos separados (no JSON), para poder filtrar por los datos congelados directamente -por ejemplo, dirección o zona- sin tener que leer cada cotización completa. No afecta el incremento 1 (clientes y direcciones_cliente no usan snapshot); aplica a cotizaciones y ordenes_servicio en incrementos futuros. Responsable: Sofía.

#	Hallazgo	Evidencia	Impacto	Pregunta / decisión para el equipo
		shot, direccion_snapshot, sin sufijo _json).		

Decisiones para resolver inconsistencias: Se aprueba CA-68 (H1); se confirma Python 3 + sqlite3 + pytest para el incremento 1 (H3); se usa ISO 8601 con hora local (H6). H2, H4 y H5 quedan resueltos como se indica en la tabla. H7: se unifica la nomenclatura de módulos del cap. 1 a los nombres del cap. 2.5 en todo el documento. H8: se agrega generador_pdf_cotizacion a la fila de RF-14 en las tablas 2.6 y 3.9. H9: se unifican los campos snapshot de cotizaciones y ordenes_servicio usando campos separados (no JSON). H7-H9 fueron aprobados por el equipo el 29-oct-2026; no afectan el incremento 1 y se aplican al actualizar la especificación (v2.5) y en los incrementos que toquen esos capítulos y tablas.

Paso 2. Primer incremento funcional

RF/HU	Prioridad	Valor para el usuario	Dependencia	¿En incremento 1?
RF-02 / HU-02	Alta	ACL deja de buscar datos de clientes en archivos sueltos; es la base de toda cotización.	almacenamiento_datos; tablas clientes y direcciones_cliente	Sí
RF-03 / HU-03	Alta	Corregir teléfono y agregar direcciones.	RF-02 terminado	No (incremento 2; reutiliza los mismos módulos)
RF-04 / HU-04	Alta	Encontrar al cliente al cotizar.	Clientes guardados (RF-02)	No (incremento 2 o 3)
RF-06 / HU-06	Alta	Conceptos manuales sin depender de SYSCOM.	almacenamiento_datos; D07 (precio cero, límites)	No (opción B evaluada)
RF-01 / HU-01	Alta	Proteger el acceso.	D09 (hash y sesiones), interfaz web	No

Opciones evaluadas (Prompt 2)

Criterio	Opción A — RF-02 Registrar cliente	Opción B — RF-06 Conceptos manuales
RF/HU	RF-02 / HU-02	RF-06 / HU-06
Valor demostrable	Cliente con dos direcciones guardado y recuperado tras reabrir la BD.	Concepto manual guardado con precio en centavos.
Módulos	almacenamiento_datos, clientes_registro	almacenamiento_datos, conceptos_manuales
Dependencias abiertas	D07 parcial (teléfono), D08 (fecha), D09 parcial	D07 (precio cero y límites), D02 (moneda) y D01 (redondeo) influyen en precios
Riesgo	Bajo: reglas simples ya definidas en CA-04, CA-38 y CA-05.	Medio: varias reglas de precio siguen pendientes con ACL.
Prueba observable	PT-02-N/L/E y P01.	PT-06-N/L/E.

Incremento elegido: Opción A (RF-02). Justificación: (1) es el orden ya sugerido en la especificación v2.4 (almacenamiento_datos → clientes_registro, prueba P01); (2) sin clientes no hay cotización (RF-07) ni orden de servicio (RF-16); (3) es la opción con menos decisiones comerciales abiertas; (4) obliga a construir y probar la capa de datos (transacciones, FK, consultas parametrizadas) que usarán todos los demás módulos.

Expresamente fuera del primer incremento

- Interfaz web y framework (dependen de D09); la entrada se simula con pruebas y un script de demostración.
- RF-01 login, RF-03 edición, RF-04 búsqueda y RF-21 configuración de ACL.
- Aviso o bloqueo por teléfono duplicado (CA-06, D07).
- Las otras siete tablas (usuarios, conceptos, cotizaciones, detalle_cotizacion, ordenes_servicio, configuracion_acl, sincronizaciones_syscom).
- registro_eventos, SYSCOM, PDF, WhatsApp, respaldos (D10).

Paso 3. Módulos y dependencias

Módulo	Responsabilidad	Necesita de	Entrega a	Prioridad
almacenamiento_datos proyecto/infraestructura/ almacenamiento_datos.py	Único acceso a la BD: crear esquema, abrir conexión con FK activas, guardar cliente y direcciones en una transacción, consultar cliente por id.	data/presupacl.db; esquema de tablas clientes y direcciones_cliente (diccionario 3.4)	clientes_registro (y después todos los módulos)	1
clientes_registro proyecto/clientes/clientes_registro.py	Validar nombre, teléfono y 1+ direcciones; pedir el guardado; devolver id_cliente o la lista de campos faltantes.	Contrato de almacenamiento_datos (puede simularse mientras T-04 no está lista)	Pruebas y, más adelante, la pantalla de clientes, clientes_busqueda y cotizaciones_creacion	2
Pruebas proyecto/tests/test_almacenamiento_datos.py proyecto/tests/test_clientes_registro.py	Verificar CA-04, CA-38, CA-05, CA-68 y P01, P11, P12 y P13.	Ambos módulos; BD temporal de prueba	Evidencia del incremento	2
clientes_busqueda, auth, interfaz	Fuera del incremento 1.	RF-02 terminado; D09	—	3

Dependencia de ejecución vs. orden de trabajo: clientes_registro necesita a almacenamiento_datos para funcionar, pero no para empezar. Con el contrato de T-02, Sofía puede programar y probar la validación (T-05) con un almacenamiento simulado mientras Vanesa construye el esquema y la transacción (T-03 y T-04). Las dos ramas se unen en T-06.

Flujo de dependencias

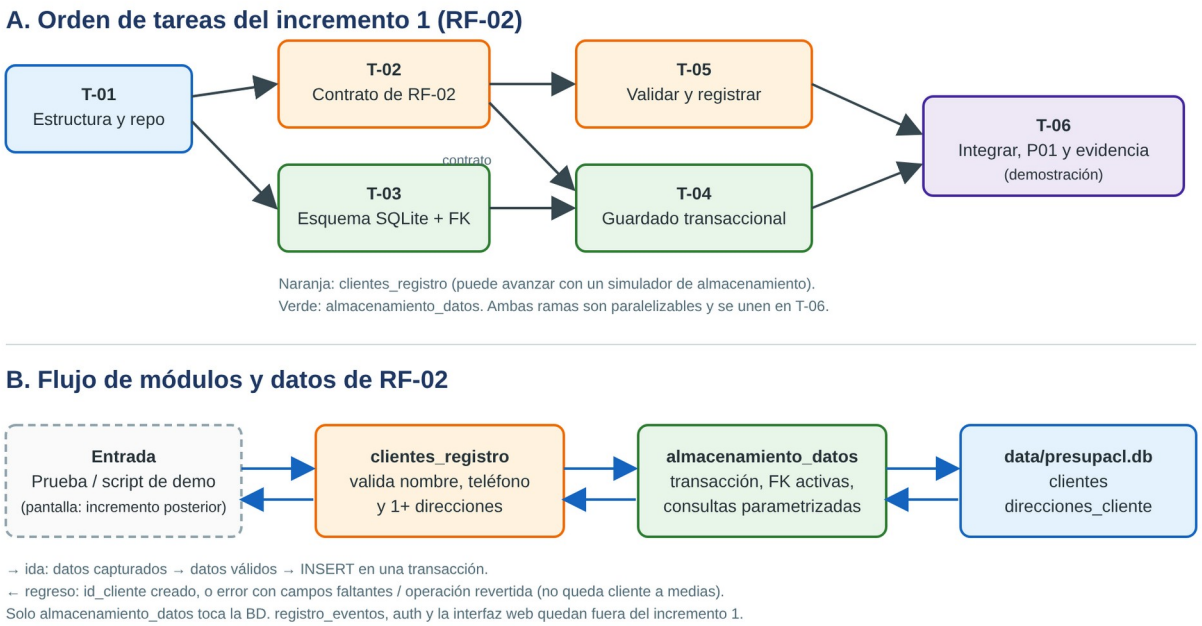


Figura 1. Arriba: orden de tareas. Abajo: flujo de módulos y datos de RF-02. Coincide con la arquitectura de la sección 2.7 de v2.4; no se agregan módulos.

Orden acordado y por qué: T-01 (estructura y repositorio) va primero porque todo commit y prueba depende de él. T-02 fija las entradas, salidas y errores antes de programar, para que ambas integrantes trabajen contra el mismo contrato. T-03 → T-04 construyen la capa de datos; T-05 la regla de negocio. T-06 integra, ejecuta P01 y registra la evidencia.

Paso 4. Tareas técnicas

ID	Tarea técnica concreta	RF/HU	Módulo / archivo	Predecesora	Evidencia de terminado
T-01	Crear la estructura base del proyecto y el repositorio	RF-02 (técnica)	proyecto/, data/, logs/, README.md, .gitignore	Ninguna	Árbol de carpetas + primer commit
T-02	Definir el contrato de entradas, salidas y errores de RF-02	RF-02 / HU-02	Docstrings en clientes_registro.py y almacenamiento_datos.py	T-01	Contrato revisado por ambas + commit
T-03	Crear el esquema de clientes y direcciones_cliente con FK activas	RF-02	infraestructura/almacenamiento_datos.py	T-01	Prueba P11 aprobada + commit
T-04	Guardar cliente y direcciones en una transacción y consultarlo por id	RF-02 / HU-02	infraestructura/almacenamiento_datos.py	T-02, T-03	Pruebas P12 y P13 aprobadas + commit
T-05	Validar campos obligatorios y registrar el cliente	RF-02 / HU-02	clientes/clientes_registro.py	T-02	PT-02-E y PT-02-E2 aprobadas (con simulador) + commit

ID	Tarea técnica concreta	RF/HU	Módulo / archivo	Predecesora	Evidencia de terminado
T-06	Integrar módulos, ejecutar pruebas y P01, y registrar evidencia	RF-02 / HU-02	tests/, matriz y bitácora	T-04, T-05	PT-02-N/L/E/E2 y P01 aprobadas, salida guardada, commit etiquetado

Fichas de tarea

T-01 · Crear la estructura base del proyecto y el repositorio

RF al que contribuye	RF-02 (necesidad técnica: sin estructura no hay dónde guardar código, pruebas ni commits). Apoya RNF-04 y RNF-07.
Módulo responsable	Proyecto (estructura de carpetas de la sección 2.8).
Resultado concreto	Repositorio con proyecto/{infraestructura,clientes,tests}/ y sus __init__.py, data/, logs/, README.md y .gitignore. Se crean solo las carpetas que usa este incremento; las demás se agregan cuando haya código.
Entradas	Estructura 2.8 de v2.4; tecnología confirmada (H3).
Salidas	Repositorio remoto con primer commit; README con cómo ejecutar pruebas.
Dependencias	Ninguna. Requiere decidir dónde se aloja el repositorio.
CA y pruebas	No implementa CA. Verificación: pytest se ejecuta sin errores aunque aún no haya pruebas.
Reglas	.gitignore excluye data/*.db, logs/*.log y .env: ninguna BD real ni credencial entra al repositorio (RNF-03).
Evidencia	Captura del árbol de carpetas + enlace al primer commit.

T-02 · Definir el contrato de entradas, salidas y errores de RF-02

RF al que contribuye	RF-02 / HU-02.
Módulo responsable	clientes_registro y almacenamiento_datos (solo firmas y docstrings, sin lógica).
Resultado concreto	Firmas acordadas: registrar_cliente(nombre, telefono, direcciones) → id_cliente o error de validación con la lista de campos faltantes; crear_cliente_con_direcciones(nombre, telefono, direcciones) → id_cliente; obtener_cliente(id_cliente) → cliente con sus direcciones o “no existe”.
Entradas	RF-02, CA-04, CA-38, CA-05, CA-68; diccionario de clientes y direcciones_cliente (3.4).
Salidas	Contrato escrito en el código y en el README; mensajes de error en español.
Dependencias	T-01.
CA y pruebas	Define los datos de PT-02-N, PT-02-L, PT-02-E y PT-02-E2.
Reglas	SUPUESTO — validar con ACL (D07): nombre y teléfono se recortan de espacios; vacío o solo espacios cuenta como faltante. Teléfono se guarda como texto, sin formato obligatorio ni unicidad.
Evidencia	Revisión cruzada (una escribe, la otra aprueba) + commit.

T-03 · Crear el esquema de clientes y direcciones_cliente con FK activas

RF al que contribuye	RF-02 (base para RF-03, RF-04 y RF-16).
Módulo responsable	almacenamiento_datos — proyecto/infraestructura/almacenamiento_datos.py.

Resultado concreto	Función que abre la BD con PRAGMA foreign_keys = ON y crea las tablas si no existen: clientes (id_cliente, nombre, telefono, creado_en, actualizado_en) y direcciones_cliente (id_direccion, id_cliente FK, direccion). Campos obligatorios como NOT NULL.
Entradas	Diccionario de datos 3.4 de v2.4; ruta de BD por parámetro (data/presupacl.db en uso real, archivo temporal en pruebas).
Salidas	Archivo SQLite con dos tablas; conexión lista para usar.
Dependencias	T-01.
CA y pruebas	P11 (FK activas). Prueba en test_almacenamiento_datos.py.
Evidencia	P11 aprobada; consulta del esquema (sqlite3 .schema) + commit.

T-04 · Guardar cliente y direcciones en una transacción y consultarlo por id

RF al que contribuye	RF-02 / HU-02.
Módulo responsable	almacenamiento_datos.
Resultado concreto	crear_cliente_con_direcciones inserta la cabecera y todas las direcciones en una sola transacción, con consultas parametrizadas; si algo falla, revierte todo. obtener_cliente devuelve el cliente y sus direcciones. Registra creado_en y actualizado_en.
Entradas	Datos ya validados por clientes_registro; contrato T-02.
Salidas	id_cliente nuevo, o excepción sin datos parciales.
Dependencias	T-02, T-03.
CA y pruebas	P12 (fallo simulado → sin cliente a medias), P13 (texto con comilla se guarda literal). Soporta CA-04 y CA-38.
Reglas	Regla 2 del proyecto: solo este módulo toca la BD, con FK y transacciones. No se crea función para borrar clientes.
Evidencia	P12 y P13 aprobadas + commit.

T-05 · Validar campos obligatorios y registrar el cliente

RF al que contribuye	RF-02 / HU-02.
Módulo responsable	clientes_registro — proyecto/clientes/clientes_registro.py.
Resultado concreto	registrar_cliente valida nombre, teléfono y al menos una dirección no vacía; si falta algo, devuelve todos los campos faltantes juntos y no llama al almacenamiento; si todo es válido, pide el guardado y devuelve el id_cliente.
Entradas	Contrato T-02; almacenamiento simulado (en pruebas unitarias) o real (en T-06).
Salidas	id_cliente o error de validación.
Dependencias	T-02 (puede terminar antes que T-04 gracias al simulador).
CA y pruebas	CA-05 → PT-02-E; CA-68 → PT-02-E2.
Evidencia	Pruebas unitarias aprobadas + commit.

T-06 · Integrar módulos, ejecutar pruebas y P01, y registrar evidencia

RF al que contribuye	RF-02 / HU-02. Cumple RNF-07 (prueba de éxito y de error).
Módulo responsable	clientes_registro + almacenamiento_datos; pruebas en proyecto/tests/.
Resultado concreto	clientes_registro usa el almacenamiento real; se ejecutan PT-02-N, PT-02-L, PT-02-E, PT-02-E2, P01, P11, P12 y P13 contra una BD temporal con datos ficticios.
Entradas	T-04 y T-05 terminadas.

Salidas	Salida de pytest guardada; registro de P01 llenado (fecha, responsable, resultado real, commit); matriz y bitácora de IA actualizadas.
Dependencias	T-04, T-05.
CA y pruebas	CA-04, CA-38, CA-05, CA-68; P01.
Evidencia	Captura de la ejecución + commit etiquetado “incremento-1”. El RF-02 se da por terminado solo con estas pruebas aprobadas.

Paso 5. Persistencia y configuración

Decisión	Respuesta del equipo
¿Qué datos serán temporales?	Los datos capturados antes de pulsar guardar y los mensajes de validación. Las BD creadas por las pruebas (se borran al terminar).
¿Qué datos deben mantenerse tras cerrar el sistema?	Cliente (nombre, teléfono, creado_en, actualizado_en) y todas sus direcciones.
¿Qué solución se eligió y por qué?	SQLite local en data/presupacl.db (Ruta B, sección 3.1 de v2.4, confirmada el 15-oct): relaciones 1 a N, transacciones y sin servidor adicional. El archivo no se comparte por red; las otras PC entran por navegador al servidor.
¿Qué módulo accede a esos datos?	Solo almacenamiento_datos. clientes_registro nunca escribe SQL.
¿Qué estructura inicial hace falta?	Dos de las nueve tablas: clientes y direcciones_cliente, con PK, FK id_cliente y NOT NULL en campos obligatorios. Las demás tablas se crean en el incremento que las use.
¿Cómo se evitará almacenar información inválida?	Doble barrera: clientes_registro valida antes (CA-05, CA-68) y la BD lo impide después (NOT NULL, FK activas). Cabecera y direcciones van en una transacción; si falla una dirección, no queda cliente a medias (P12). Consultas parametrizadas (P13).

Tareas de datos y riesgos (Prompt 5)

- **Almacenamiento:** crear esquema (T-03) y guardado transaccional (T-04).
- **Lectura:** obtener_cliente por id para comprobar P01 (T-04). La búsqueda por nombre o teléfono es RF-04, fuera del incremento.
- **Validación:** campos obligatorios y al menos una dirección (T-05).
- **Riesgo de inconsistencia:** cliente sin direcciones si se guardan por separado → se mitiga con la transacción y P12.
- **Riesgo de pérdida:** borrar o versionar por error data/presupacl.db → .gitignore y BD temporal en pruebas. El respaldo formal sigue pendiente (D10).
- **Riesgo de FK inactivas:** SQLite las trae apagadas por defecto → se activan en cada conexión y se prueban con P11.

Paso 6. Pruebas planificadas antes de programar

Todas las pruebas usan datos ficticios y una BD temporal. Ninguna se ha ejecutado todavía; el resultado real se registrará en T-06.

RF / CA · Prueba	Tipo	Datos / condición	Resultado esperado	Nivel	Evidencia
RF-02 / CA-04 · PT-02-N	Normal	Nombre “Cliente de prueba”, teléfono “4770000000”, dos direcciones.	Se devuelve id_cliente; al consultar, aparecen los datos y las dos direcciones.	Integración	Salida pytest + commit

RF / CA · Prueba	Tipo	Datos / condición	Resultado esperado	Nivel	Evidencia
RF-02 / CA-38 · PT-02-L	Límite	Mismo cliente con exactamente una dirección.	Se guarda correctamente con una dirección.	Integración	Salida pytest
RF-02 / CA-05 · PT-02-E	Error	Omitir nombre; luego omitir teléfono; luego ambos (también solo espacios).	Se indican los campos faltantes y no se guarda nada (0 filas en clientes).	Unitaria	Salida pytest
RF-02 / CA-68 · PT-02-E2	Error	Nombre y teléfono válidos, lista de direcciones vacía o con una dirección en blanco.	Se indica que falta al menos una dirección y no se guarda nada.	Unitaria	Salida pytest
RF-02, RF-03 · P01	Normal	Crear cliente con dos direcciones, cerrar la conexión y abrir otra sobre el mismo archivo.	El cliente y sus dos direcciones siguen presentes.	Integración	Registro P01 (fecha, responsable, commit)
RF-02 · P11 (nueva)	Error	Insertar una dirección con id_cliente inexistente.	La BD la rechaza (FK activa).	Unitaria	Salida pytest
RF-02 · P12 (nueva)	Error	Simular un fallo al guardar la segunda dirección.	Se revierte todo: no queda cliente ni la primera dirección.	Unitaria	Salida pytest
RF-02 · P13 (nueva)	Límite	Nombre con comilla y caracteres especiales: “O’Neil; DROP TABLE”.	Se guarda y se lee literal; las tablas siguen intactas.	Unitaria	Salida pytest

Advertencias de verificabilidad (Prompt 6)

- **CA-04, CA-38 y CA-05** son verificables con datos concretos y ya tienen prueba (PT-02-N/L/E).
- **Faltaba** una prueba para “cero direcciones” → se agrega CA-68 / PT-02-E2, aprobado por el equipo.
- **CA-06** (teléfono duplicado) no es verificable mientras no exista la regla de ACL (D07); no se prueba en este incremento.
- P11, P12 y P13 son pruebas técnicas de persistencia que continúan la numeración P01–P10; no agregan funcionalidad, comprueban las reglas de integridad ya definidas.

Paso 7. Plan de ejecución

Esfuerzo relativo (B = bajo, M = medio, A = alto). No se fijan fechas ni horas porque no se proporcionaron; el reparto fue acordado por el equipo. Cada tarea tiene una responsable y la otra integrante revisa.

Orden	Tarea	Responsable	Revisa	Esfuerzo	Entrega / hito	Estado
1	T-01	Vanesa	Sofía	B	H1 · Repositorio y estructura listos	Pendiente
2	T-02	Sofía	Vanesa	B	Contrato acordado	Pendiente
3a	T-03	Vanesa	Sofía	M	Esquema con FK probado (P11)	Pendiente
3b	T-05	Sofía	Vanesa	M	Validación probada con simulador	Pendiente
4	T-04	Vanesa	Sofía	M	H2 · Guardado transaccional probado (P12, P13)	Pendiente
5	T-06	Ambas	Ambas	M	H3 · RF-02 demostrable (PT-02-* y P01)	Pendiente

Lógica del orden (Prompt 7): T-01 y T-02 son cuellos de botella cortos y van primero. Después, 3a y 3b corren en paralelo: cada integrante tiene una rama propia y la carga queda equilibrada (Vanesa: datos; Sofía: reglas). T-04 es la tarea con más riesgo técnico (transacción y rollback), por eso tiene revisión cruzada. T-06 se hace en pareja para que ambas puedan explicar el incremento completo.

Paso 8. Riesgos, cambios y uso de IA

Riesgo / supuesto	Impacto	Acción preventiva	Responsable
Framework web aún sin elegir (D09).	Medio: la pantalla de clientes no puede iniciar hasta decidirlo.	Lenguaje confirmado (Python 3 + sqlite3 + pytest). El incremento 1 no usa framework web; se elige antes del incremento que agregue pantallas.	Ambas
Regla de teléfono (D07) sin definir.	Medio: podría cambiar la validación.	Validación aislada en clientes_registro; cambiarla solo afecta T-05 y sus pruebas.	Sofía
Formato de fechas (D08).	Bajo.	SUPUESTO ISO 8601 hora local; un solo punto del código genera la fecha.	Vanesa
La IA genera más código del pedido (otras tablas, pantallas).	Medio: alcance inflado y código sin revisar.	Pedir un módulo a la vez, explicar antes de generar, revisar cada línea antes del commit.	Ambas
BD real o datos personales en el repositorio.	Alto (privacidad).	.gitignore para data/*.db y .env; solo datos ficticios en pruebas.	Vanesa
Repositorio inexistente al iniciar.	Medio: sin evidencia de commits.	Primera tarea T-01; anotar el enlace en la portada.	Vanesa
Nomenclatura de módulos (cap. 1), mapeo de RF-14 y formato de snapshot (H7-H9). Resuelto: aplicados en la especificación v2.5 el 3-nov-2026.	Bajo para el incremento 1 (no usa esos capítulos ni esas tablas); afecta incrementos futuros si no se actualiza a tiempo.	Hecho: especificación v2.5 actualizada con H7, H8 y H9, además de CA-68 y P11-P13.	Sofía

Revisión crítica del plan (Prompt 8)

Nivel	Observación	Acción propuesta
Bloqueante (resuelto)	Confirmar la tecnología (D09) antes de T-01.	Resuelto: Python 3 + sqlite3 + pytest confirmados por el equipo para el incremento 1.
Importante (resuelto)	CA-68 es un criterio nuevo derivado de RF-02.	Aprobado; se añade a la matriz de trazabilidad en v2.5.
Importante	El documento v2.4 dice “cerrar D01–D11 antes de programar”.	Aclarar en v2.5 que cada incremento solo exige cerrar las D que lo afectan.
Mejora opcional	Un script de demostración por consola haría P01 más visible en clase.	Opcional dentro de T-06; no agrega funcionalidad.

Verificación de huecos: no hay tareas huérfanas (todas apuntan a RF-02), el único RF del incremento tiene tareas, todos los CA del incremento tienen prueba, no hay dependencias circulares (T-01 → T-02/T-03 → T-04/T-05 → T-06) y ninguna tarea es desproporcionada.

Bitácora de uso de IA

Registro de las consultas de esta sesión de planeación (29-oct-2026). El equipo revisó cada propuesta; se anota qué se aceptó y qué se cambió.

Paso	Consulta / prompt usado	Propuesta útil	Qué aceptamos o cambiamos	Responsable
1	Prompt 1: revisión de consistencia de v2.4 sin agregar funciones.	Seis hallazgos; propuesta de CA-68.	Aceptamos los seis hallazgos y CA-68. Cambiamos: H4 se resuelve cerrando solo las D que afectan cada incremento. Además, una segunda revisión del mismo Paso 1 (ejecutada hoy sobre el documento ya entregado) encontró tres hallazgos adicionales de consistencia documental (H7-H9: nomenclatura de módulos, mapeo RF-14→módulo, formato de los campos snapshot); el equipo aprobó los tres el 29-oct y no afectan el incremento 1.	Sofía y Vanesa
2	Prompt 2: dos opciones de primer incremento.	Opción A (RF-02) vs. B (RF-06); recomienda A.	Aceptamos la opción A. Cambiamos: RF-03 pasa al incremento 2 para mantener un solo RF.	Sofía y Vanesa
3	Prompt 3: secuencia de módulos y dependencias.	Dos ramas paralelas con contrato y simulador.	Aceptamos el orden y las ramas paralelas; sin cambios a la arquitectura.	Vanesa
4	Prompt 4: desglose en tareas con ficha mínima.	T-01 a T-06 con supuestos marcados.	Aceptamos las seis tareas. Cambiamos: T-01 crea solo las carpetas que usa el incremento.	Sofía
5	Prompt 5: tareas de datos y riesgos.	Esquema de 2 tablas, transacción, FK; P11–P13.	Aceptamos crear solo 2 de las 9 tablas y las pruebas P11–P13.	Vanesa
6	Prompt 6: pruebas trazables sin código.	Ocho casos ligados a CA; aviso sobre CA-06.	Aceptamos los ocho casos. CA-06 queda fuera hasta que ACL defina la regla (D07).	Sofía
7	Prompt 7: secuencia, hitos y cargas.	Hitos H1–H3 y reparto equilibrado.	Aceptamos hitos y reparto; no fijamos fechas exactas.	Sofía y Vanesa
8	Prompt 8: revisión crítica.	Un bloqueante (D09), dos importantes.	Resolvimos el bloqueante (D09) y aprobamos CA-68; la demo por consola queda como opcional.	Sofía y Vanesa

Paso 9. Verificación final y aprobación

Se marcan solo los puntos que el documento demuestra. Plan revisado y aprobado por el equipo el 29 de octubre de 2026.

Estado	Punto de verificación	Dónde se demuestra / acción pendiente
☑️	Todos los RF del primer incremento están incluidos en una o más tareas.	RF-02 en T-01 a T-06 (Paso 4).
☑️	Cada tarea importante tiene responsable, dependencia y evidencia de terminado.	Pasos 4 y 7. Reparto acordado.
☑️	Los criterios de aceptación del incremento cuentan con pruebas diseñadas.	Paso 6. CA-68 aprobado.
☑️	Los módulos y archivos previstos coinciden con la arquitectura.	Paso 3; rutas de la sección 2.8 de v2.4.
☑️	La estrategia de persistencia está reflejada en tareas concretas.	T-03 y T-04; Paso 5.
☑️	El plan distingue lo incluido y excluido del primer incremento.	Paso 2.
☑️	Los riesgos bloqueantes tienen acción y responsable.	Paso 8. El único bloqueante (D09) se resolvió para el incremento 1.
☑️	Las sugerencias de IA fueron revisadas por el equipo.	Bitácora de uso de IA: qué se aceptó y qué se cambió en cada paso.

Decisiones pendientes y cómo se resolverán	D09: lenguaje confirmado para el incremento 1; framework web, biblioteca PDF, hash y sesiones se deciden antes de los incrementos que los usan. D07 (teléfono) y D08 (zona horaria): consultar con ACL; mientras tanto se usan los supuestos marcados en T-02 y H6. D01–D06 y D10 no afectan este incremento y siguen abiertas.
Módulo y tarea con la que comenzaremos	T-01 (estructura y repositorio), seguida de almacenamiento_datos con T-03.
Primera demostración que presentaremos	Ejecución de PT-02-N, PT-02-L, PT-02-E, PT-02-E2 y P01: un cliente con dos direcciones sobrevive al cierre de la BD; un registro sin nombre, teléfono o dirección se rechaza sin guardar nada.

Control de cambios respecto a v2.4

Tipo	Elemento	Detalle
➕ Agrega	CA-68 · Error (RF-02, HU-02)	Dado nombre y teléfono válidos sin ninguna dirección, cuando intenta guardar, entonces se indica que falta al menos una dirección y no se guarda. Prueba PT-02-E2. Aprobado por el equipo el 29-oct-2026; incorporado en la especificación v2.5 (sección 1.3).
➕ Agrega	P11, P12, P13	Pruebas técnicas de persistencia: FK activas, rollback ante fallo y consultas parametrizadas. Incorporadas en la especificación v2.5 (sección 3.10).
➕ Agrega	T-01 a T-06	Tareas del incremento 1.
✅ Sin cambio	RF-02, HU-02, CA-04, CA-38, CA-05, PT-02-N/L/E, P01	Se usan tal como están en v2.4.
✅ Sin cambio	Módulos, carpetas y tablas	almacenamiento_datos y clientes_registro con las rutas de 2.8; tablas clientes y direcciones_cliente según 3.4.
⚠️ Conflicto	Sección 4.2/5 “cerrar D01–D11 antes de programar”	Se propone cerrar solo las D que afectan cada incremento; para RF-02: D07 parcial, D08 y D09 parcial con supuestos marcados.

Tipo	Elemento	Detalle
✏️ Modifica	Tecnología (D09)	Antes: lenguaje sin confirmar → Después: Python 3 + sqlite3 + pytest para el incremento 1. Framework, PDF, hash y sesiones siguen pendientes.
✏️ Modifica	Nomenclatura de módulos del cap. 1 (H7)	Antes: nombres de 11 módulos gruesos → Después: nombres finos del cap. 2.5 en todo el documento. Aplicado en v2.5.
✏️ Modifica	RF-14 en las tablas 2.6 y 3.9 (H8)	Antes: solo cotizaciones_edicion → Después: cotizaciones_edicion y generador_pdf_cotizacion. Aplicado en v2.5.
✏️ Modifica	Campos snapshot de cotizaciones (H9)	Antes: cliente_snapshot_json, direccion_snapshot_json y acl_snapshot_json → Después: columnas separadas, igual que ordenes_servicio. Aplicado en v2.5.